

Assignment No. 4

Aim: To implement connectivity of DHT11, Flame, IR or GAS sensor with Arduino Uno and display the results on serial monitor.

Theory:

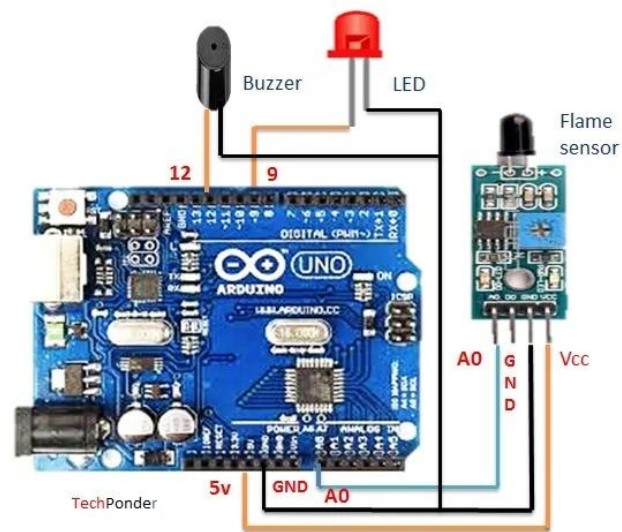
Different sensors can be interfaced with the Arduino Uno to monitor environmental conditions and display readings on the Serial Monitor:

- **DHT11 (Temperature & Humidity Sensor):** Reads temperature and humidity values and sends them to the Serial Monitor.
- **Flame Sensor:** Detects the presence of fire or heat and displays an alert message on the Serial Monitor.
- **IR Sensor:** Detects objects or movement and provides a signal output displayed as detection status.
- **Gas Sensor (MQ Series):** Measures the concentration of gases like CO, CO₂, or LPG and sends data to the Serial Monitor for monitoring air quality.

These sensors help in applications such as weather monitoring, fire detection, security systems, and air quality monitoring.

Flame Sensor

Diagram:



Code:

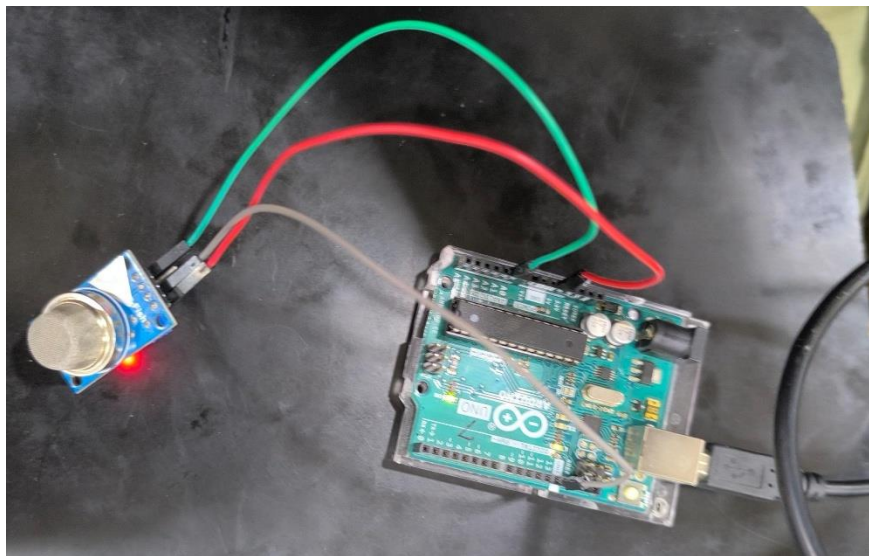
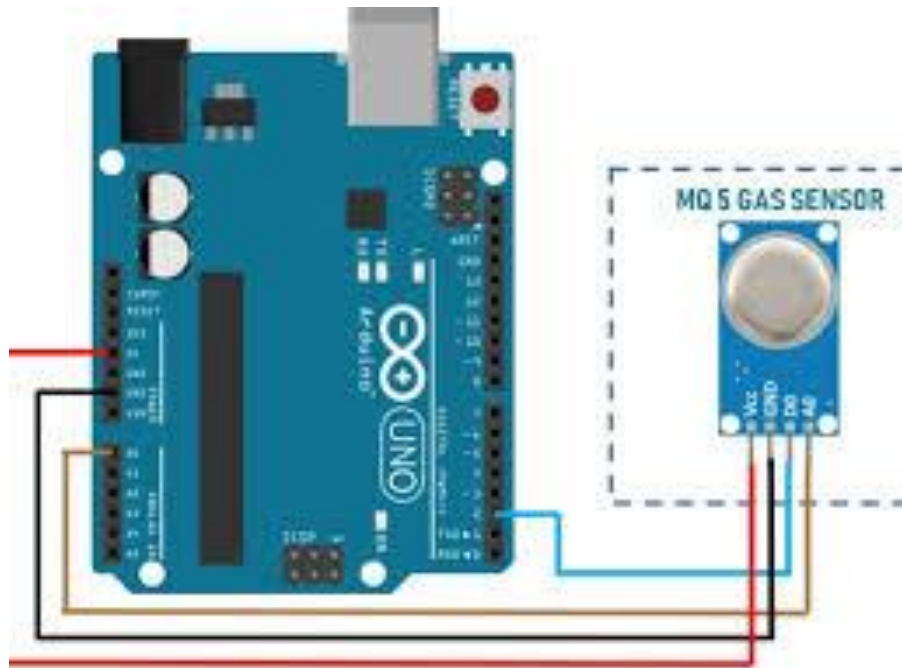
```
Arduino Uno
sketch_jan28a.ino
1  #include<SoftwareSerial.h>
2  int sensorPin = A0; // select the input pin for the LDR
3  int sensorValue = 0; // variable to store the value coming from the sensor
4  int led = 9; // Output pin for LED
5  int buzzer = 12; // Output pin for Buzzer
6  void setup() {
7  // declare the ledPin and buzzer as an OUTPUT:
8  pinMode(led, OUTPUT);
9  pinMode(buzzer,OUTPUT);
10 Serial.begin(9600);
11 }
12 void loop()
13 {
14 Serial.println("Welcome!");
15 sensorValue = analogRead(sensorPin);
16 Serial.println(sensorValue);
17 if (sensorValue < 100)
18 {
19 Serial.println("Fire Detected");
20 Serial.println("LED on");
21 digitalWrite(led,HIGH);
22 digitalWrite(buzzer,HIGH);
23 delay(1000);
24 }
25 digitalWrite(led,LOW);
26 digitalWrite(buzzer,LOW);
27 delay(sensorValue);
28 }
```

Output:

```
+|
Welcome!
1003
Welcome!
997
Welcome!
1009
Welcome!
1022
Welcome!
1023
Welcome!
1023
Welcome!
1023
Welcome!
24
Fire Detected
LED on
Welcome!
23
Fire Detected
LED on
Welcome!
31
Fire Detected
LED on
```

Gas Sensor

Diagram:



Code:

```
Arduino Uno
sketch_jan28a.ino
1  const int sensorPin = A0;          // Pin connected to the gas sensor
2  const int sensorThreshold = 300;   // Threshold value for gas concentration
3
4  void setup() {
5      // Initialize serial communication
6      Serial.begin(9600);
7      Serial.println("Gas Sensor Monitoring Started");
8
9      // Set the sensor pin as input
10     pinMode(sensorPin, INPUT);
11 }
12
13 void loop() {
14     // Read the gas sensor value
15     int sensorValue = analogRead(sensorPin);
16
17     // Print the gas concentration value
18     Serial.print("Gas Concentration Value: ");
19     Serial.println(sensorValue);
20
21     // Check if the gas concentration exceeds the threshold
22     if (sensorValue > sensorThreshold) {
23         Serial.println("Warning: Gas concentration too high!");
24     } else {
25         Serial.println("Gas concentration is within control.");
26     }
27
28     // Add a delay to avoid flooding the serial monitor
29     delay(500); // 500 milliseconds
30 }
31
```

Output:

```
Warning: Gas concentration too high!
Gas Concentration Value: 406
Warning: Gas concentration too high!
Gas Concentration Value: 421
Warning: Gas concentration too high!
Gas Concentration Value: 422
Warning: Gas concentration too high!
Gas Concentration Value: 420
Warning: Gas concentration too high!
Gas Concentration Value: 414
Warning: Gas concentration too high!
Gas Concentration Value: 423
Warning: Gas concentration too high!
Gas Concentration Value: 435
Warning: Gas concentration too high!
```

```
Gas concentration is within control.
Gas Concentration Value: 262
Gas concentration is within control.
Gas Concentration Value: 266
Gas concentration is within control.
Gas Concentration Value: 272
Gas concentration is within control.
Gas Concentration Value: 278
Gas concentration is within control.
Gas Concentration Value: 284
Gas concentration is within control.
Gas Concentration Value: 288
Gas concentration is within control.
Gas Concentration Value: 293
Gas concentration is wi
```

DHT11 Moisture Sensor

Diagram:

Code:

```
// Define the pin for the moisture/water sensor
const int moistureSensorPin = A0; // Analog pin connected to sensor

void setup() {
  // Start the serial communication
  Serial.begin(9600);
}

// For zero moisture, we get maximum value of 10-bit ADC, i.e. 1023.
void loop() {
  // Read the value from the moisture/water sensor
  int moistureLevel = analogRead(moistureSensorPin);

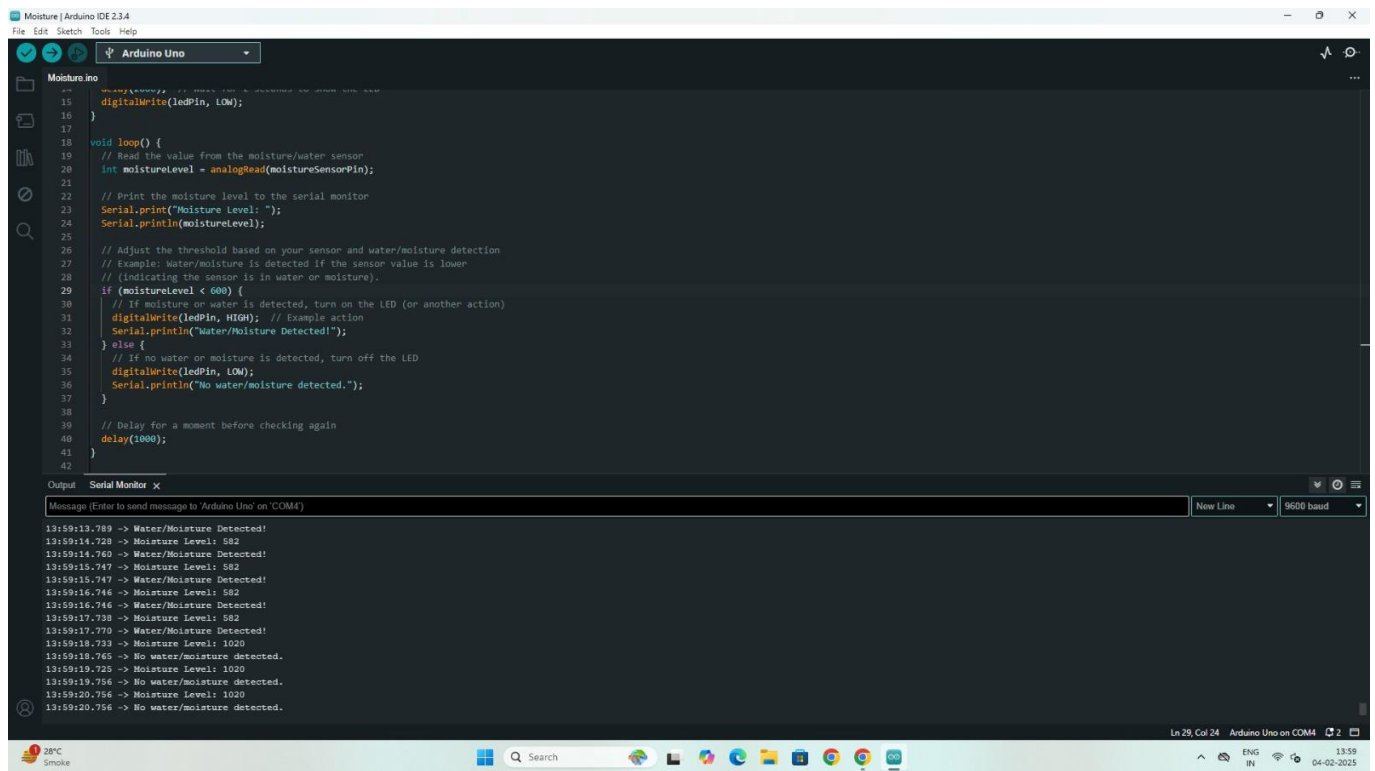
  // Print the moisture level to the serial monitor
  Serial.print("Moisture Level: ");
  Serial.println(moistureLevel);

  if (moistureLevel < 600) {
    // If moisture or water is detected, print message to serial monitor
    Serial.println("Water/Moisture Detected!");
  } else {
    // If no water or moisture is detected, print message to serial monitor
    Serial.println("No water/moisture detected.");
  }
}
```



```
// Delay for a moment before checking again  
  
delay(1000);  
  
}
```

Output:

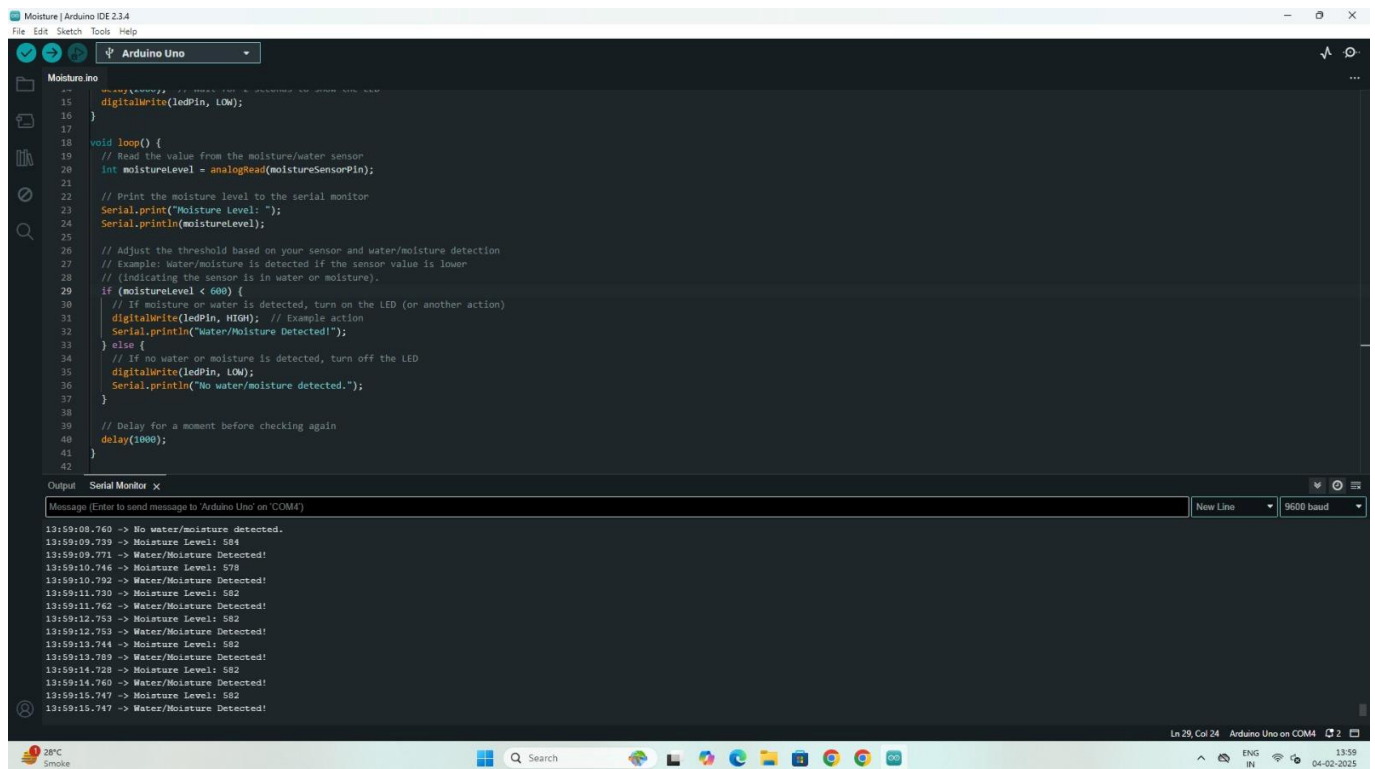


The screenshot shows the Arduino IDE with the following code in the editor:

```
15 digitalWrite(ledPin, LOW);  
16 }  
17  
18 void loop() {  
19   // Read the value from the moisture/water sensor  
20   int moistureLevel = analogRead(moistureSensorPin);  
21  
22   // Print the moisture level to the serial monitor  
23   Serial.print("Moisture Level: ");  
24   Serial.println(moistureLevel);  
25  
26   // Adjust the threshold based on your sensor and water/moisture detection  
27   // Example: Water/moisture is detected if the sensor value is lower  
28   // (indicating the sensor is in water or moisture).  
29   if (moistureLevel < 600) {  
30     // If moisture or water is detected, turn on the LED (or another action)  
31     digitalWrite(ledPin, HIGH); // Example action  
32     Serial.println("Water/Moisture Detected!");  
33   } else {  
34     // If no water or moisture is detected, turn off the LED  
35     digitalWrite(ledPin, LOW);  
36     Serial.println("No water/moisture detected.");  
37   }  
38  
39   // Delay for a moment before checking again  
40   delay(1000);  
41 }  
42
```

The Serial Monitor output shows the following messages:

```
13:59:13.789 -> Water/Moisture Detected!  
13:59:14.728 -> Moisture Level: 582  
13:59:14.760 -> Water/Moisture Detected!  
13:59:15.747 -> Moisture Level: 582  
13:59:15.747 -> Water/Moisture Detected!  
13:59:16.746 -> Moisture Level: 582  
13:59:16.746 -> Water/Moisture Detected!  
13:59:17.738 -> Moisture Level: 582  
13:59:17.770 -> Water/Moisture Detected!  
13:59:18.733 -> Moisture Level: 1020  
13:59:18.765 -> No water/moisture detected.  
13:59:19.725 -> Moisture Level: 1020  
13:59:19.756 -> No water/moisture detected.  
13:59:20.756 -> Moisture Level: 1020  
13:59:20.756 -> No water/moisture detected.
```



The screenshot shows the Arduino IDE with the following code in the editor:

```
15 digitalWrite(ledPin, LOW);  
16 }  
17  
18 void loop() {  
19   // Read the value from the moisture/water sensor  
20   int moistureLevel = analogRead(moistureSensorPin);  
21  
22   // Print the moisture level to the serial monitor  
23   Serial.print("Moisture Level: ");  
24   Serial.println(moistureLevel);  
25  
26   // Adjust the threshold based on your sensor and water/moisture detection  
27   // Example: Water/moisture is detected if the sensor value is lower  
28   // (indicating the sensor is in water or moisture).  
29   if (moistureLevel < 600) {  
30     // If moisture or water is detected, turn on the LED (or another action)  
31     digitalWrite(ledPin, HIGH); // Example action  
32     Serial.println("Water/Moisture Detected!");  
33   } else {  
34     // If no water or moisture is detected, turn off the LED  
35     digitalWrite(ledPin, LOW);  
36     Serial.println("No water/moisture detected.");  
37   }  
38  
39   // Delay for a moment before checking again  
40   delay(1000);  
41 }  
42
```

The Serial Monitor output shows the following messages:

```
13:59:09.760 -> No water/moisture detected.  
13:59:09.739 -> Moisture Level: 586  
13:59:09.771 -> Water/Moisture Detected!  
13:59:10.746 -> Moisture Level: 578  
13:59:10.792 -> Water/Moisture Detected!  
13:59:11.740 -> Moisture Level: 582  
13:59:11.762 -> Water/Moisture Detected!  
13:59:12.753 -> Moisture Level: 582  
13:59:12.753 -> Water/Moisture Detected!  
13:59:13.744 -> Moisture Level: 582  
13:59:13.789 -> Water/Moisture Detected!  
13:59:14.728 -> Moisture Level: 582  
13:59:14.760 -> Water/Moisture Detected!  
13:59:15.747 -> Moisture Level: 582  
13:59:15.747 -> Water/Moisture Detected!
```

Conclusion:

Understanding different sensors, controllers, and wireless modules helps in selecting the right components for IoT, automation, and robotics projects. Controllers like Arduino and Raspberry Pi provide flexible platforms for development, while communication modules like WiFi and GSM enable wireless connectivity, making them essential for smart applications.